

Esercizio 2 (11 punti).

Considerare l'architettura RISC-V a ciclo singolo nella figura in basso (e in allegato).

Si vuole aggiungere alla CPU l'istruzione **load word if greater** (lwig), di **tipo I** e sintassi assembly:

lwig outreg, val1, val2_address

(dove outreg, val1 e val2_address sono rispettivamente i campi rd, rs1, e il campo immediate dell'istruzione). L'istruzione deve operare come segue:

a) carica dal banco registri val1;

b) estenda il segno di **val2_address** ottenendo una word che chiameremo **val2_address₃₂**

c) carica da memoria la word all'indirizzo (**val2_address₃₂ × 4**) + **0x1001 0000**; nel seguito, chiameremo questa word **val2**;

d) compara **val1** (ottenuto dal banco registri) a **val2** (letto da memoria);

e) se **val1 ≥ val2** salva nel registro di indice **outreg** il valore **val1**; altrimenti, salva nel registro di indice **outreg** il valore **val2**.

Esempio: Supponiamo che **val1** sia s4, contenente 0x00AA 0FD6, che **outreg** sia t4,
che **val2_address** valga 0x0203 e che in memoria, all'indirizzo **0x1001 080C**, ci sia la word 0x00AA 00DA.

In tal caso, abbiamo i seguenti risultati:

val1 = 0x00AA 0FD6

(0x0203 × 4) + 0x1001 0000 = 0x1001 080C. Sappiamo (in questo esempio) che la word salvata in memoria a quell'indirizzo: è 0x00AA 00DA.

Dunque, **val2** = 0x00AA 00DA. Ne consegue che **val1 > val2** Dunque, viene salvato val1 nel registro t4, ossia t4 = 0x00AA 0FD6.

1) Mostrare le **modifiche all'architettura** della CPU RISC-V, avendo cura di aggiungere eventuali altri componenti necessari a realizzare l'istruzione. A tal fine, si può alterare la stampa del diagramma architetturale oppure ridisegnare i componenti interessati dalla modifica, avendo cura di indicare i fili di collegamento e tutti i segnali entranti ed uscenti. Indicare inoltre sul diagramma i **segnali di controllo** che la CU genera *per realizzare l'istruzione*.

2) Indicare il contenuto in bit della word che esprime l'istruzione

lwig t4, s4, 0x0203

compilando la tabella sottostante (assumiamo che lo *OpCode* di wvss sia 0x3D, e che porzioni inutilizzate dall'istruzione siano codificate con zeri).

0	0	1	0	0	0	0	0	0	0	0	0	1	1	1	0	1	0	0	0	0	0	0	1	1	1	0	1	0	1	1	1	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

3) Supponendo che l'accesso alle **memorie** impieghi **200 ns**, l'accesso ai **registri** **50 ns**, le operazioni dell'**ALU** e dei **sommatori** **150 ns**, e che gli altri ritardi di propagazione dei segnali siano trascurabili, indicare la durata totale del **ciclo di clock** tale che anche l'esecuzione della nuova istruzione sia permessa *spiegando i calcoli effettuati*.

il percorso più lungo è MEM + ALU + MEM + SUB + REG = 200 + 150 + 200 + 150 + 50 = 750 (il sub deve attendere il risultato di mem)

4) Indicando con Lwig il segnale di controllo che viene asserito per eseguire la nuova istruzione, assumiamo che

a) tutti i segnali di tipo *don't care* siano pari a 0 e che

b) la Control Unit della CPU RISC-V modificata per supportare Lwig sia difettosa e sovrascriva il segnale RegWrite come segue:

RegWrite = MemWrite (il simbolo = denota che la variabile a sinistra assume il valore dato della variabile a destra)

In tal caso, indicare quale valore sia assegnato a a7 al termine dell'esecuzione del seguente frammento di codice, assumendo che i registri a7, s0 e s1 siano inizializzati a 0. Motivare la propria risposta.

addi s0, s0, 0x8004

beq s1, s0, Out

```
        jal  x0, Exit
Out:    add  a7, zero, s1
Exit:   addi a7, a7, 0x8002
```

Le istruzioni `addi` darebbero errore perchè la parte immediata è più di 12 bit. Ignorando questo errore: la prima istruzione non scrive, la seconda salta, e le ultime due non scrivono quindi il risultato è 0,0,0.

